Lecture-2

Software Processes

The software process

O serie de activitati structurate necesare dezvoltarii unui sistem software

```
1. Specification
2. Design
3. Validation
4. Evolution
```

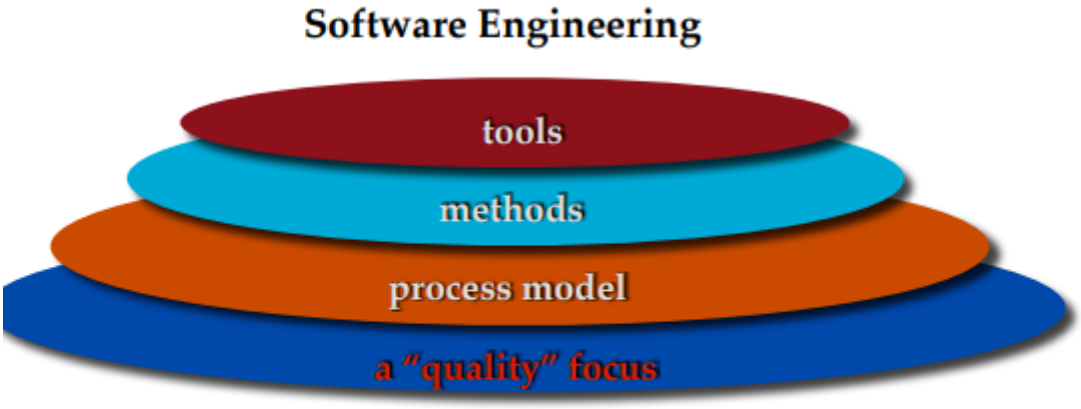
Software process model = reprezentare abstracta a unui proces (o descriere a unui proces dintr o perspectiva particulara)

Generic software process model

1. The Waterfall model (faze diferite si separate pt specificaation si development)
2. Evolutionary development - Model iterativ (Specification, development si validation sunt intercalate - > avem sprint uri)
3. Component-based software engineering(Sistemul este asamblat din componentele existente)

Concluzie
Exista doua tipuri de procese de dezvoltare software
1. Waterfall -> descompunere bazata pe activitati
2. Iterative -> descompunere bazata pe subseturi de functionalitati

A Layered View of Software Engineering



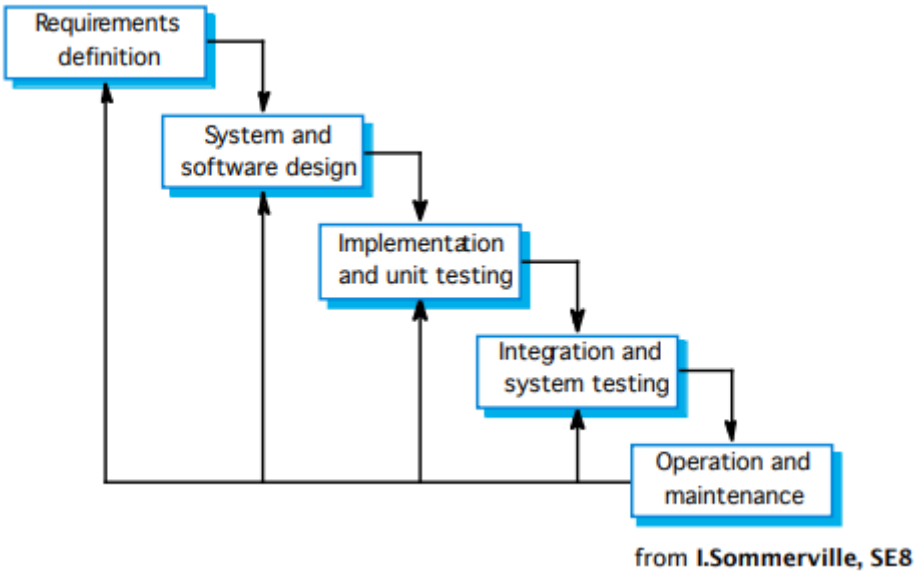
from **R.S.Pressman, 2005**

- **Quality Focus** = bedrock
- **Process** = framework and technological glue
- **Methods** = how-to (in requirement, design, construction, testing etc.)
- **Tools** = support for methods and process (CASE)

Phases of Software Engineering

- Definition Phase (**WHAT**)
  - what is processed? what performance? what interfaces? what constraints?
  - (i) information engineering; (ii) requirements eng. ; (iii) project planning
- Development Phase (**HOW**)
  - how to structure data? how to implement functions? how to choose an architecture? how to translate design into implementation?
  - (i) software design; (ii) code construction ; (iii) software testing
- Support Phase (**CHANGE**)
  - change erroneous code; change inadequate code
  - Types of change:
    1. Correction -- error in code
    2. Adaptation -- change in environment
    3. Enhancement -- change in needs

Waterfall Model



O cascada de activitati - se trece cu tot sistemul prin ele

Waterfall model phases

```
1. Definirea cerintelor
2. Proiectarea design-ului sistemului
3. Implementarea si testarea unitara (a unor module independente)
4. Integrarea si testarea sistemului (integram modulele si testam tot
sistemul)
5. Operationalizarea si mentenanta sistemului
```

Avem o bucla de feedback/ de reglaj(fara de care nu poate functiona niciun sistem)

Problema principala : Este dificil ca procesul de dezvoltare sa se adapteze schimbarilor ulterioare (abia dupa ajungerea in productie; o faza trebuie sa fie completa pana la trecerea la urmatoarea faza)

```
- Toate activitatile dureaza mult -> de la prima activitate (definirea
cerintelor) pana la ultima (ajungerea in productie) NU avem bucla de reglaj
(feedback-loop)
- Sistemul fiind intr-o lume dinamica, trebuie sa se schimbe frecvent ->
singurul moment in care poti interveni asupra cerintelor/proiectarii etc. e
mom in care sistemul ajunge operational -timpul e extrem de important ->
avem nevoia de a putea sa reglam , sa ne adaptam cel mai rapid
- Problema cea mai mare din Waterfall e ca NU pot sa acomodez schimbarea, nu
pot sa accept schimbarea in sistem sufficient de frecvent
- In momentul in care incepe procesul trebuie sa se incheie -> n-ai ce face
-activitatile sunt separate in mod rigit , secvential, fara sa ma pot
intoarce inapoi -> nepractic
```

Waterfall model problems

1. Inflexibilitatea partitionarii proiectului in etape distincte : este dificil sa raspunzi prompt la cererile in continua schimbare ale clientului
   - Este ok daca(ambele in acelasi timp)
   1. Daca cerintele au fost bine intelese de la inceput
   2. Schimbarile vor fi extrem de limitate pe parcursul procesului de proiectare
2. Ignora tot de la inceput la sfarsit
3. Uraste cuvantul schimbare

Putine afaceri au cerinte stabile, iar modelul Waterfall este folosit in sisteme mari de la proiecte ingineresti unde sistemul este dezvoltat in mai multe locatii(nu toti au cum sa vb zilnic intre ei)

```
Lucrurile sunt foarte predictibile/clare -> ex:3 luni avem pt cerinte ->
avem documentul de cerinte, parafat, nu se mai schimba nimic -> avem o tinta
bine stabilita, incepem proiectarea -> e foarte clean, riguros si usor de
urmarit -> stii mereu unde esti si ce urmeaza
In plus, procesul de dezvoltare in V (validare) : nu e complet waterfall
```

Iterative Development

```
E ca si cum ai face Waterfall pe un nano-sistem
Diferenta = timpul
```

Essence of iterations

```
1. Primim produse de calitate dupa fiecare iteratie
2. Nu exista prototipuri sau draft-uri
3. Produs real
```

Iterative Process(with Incremental delivery)

- Development and delivery is broken down into **increments**
  - each increment delivering part of the required functionality
  - instead of delivering the system as a single delivery.

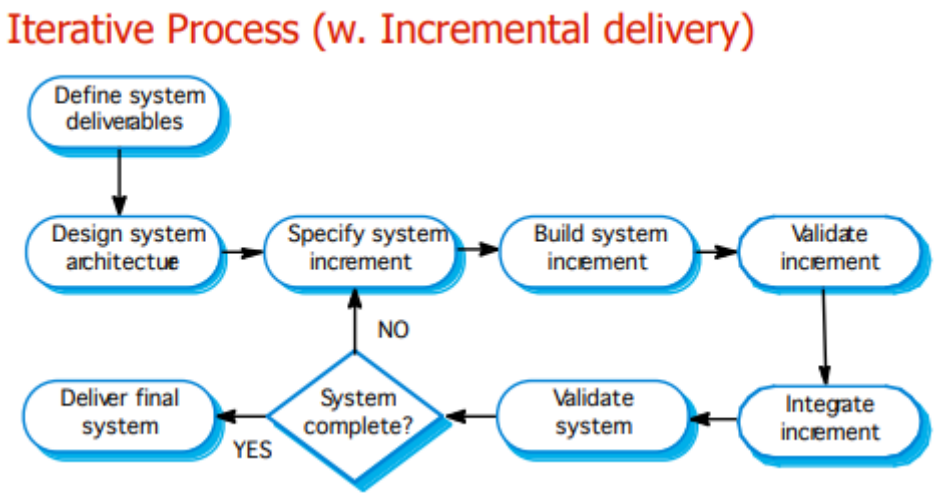
```
De unde vine numele? Se repeta acelasi tip de activitati cu fiecare feature
pe care vrem sa-l implementam. La finalul fiecarei iteratii, adica dupa ce
am executat toate activitatile pentru un feature, sistemul are un nou
```

```
feature si acesta se adauga la cele deja existente, de aceea se cheama
incremental
```

Prioritized user requirements : cele mai prioritare cerinte sunt incluse in primele iteratii efectuate

Requirements frozen : in timpul efectuarii unui increment, toate celelalte cerinte sunt inghetate, dar cerintele pentru incrementii viitori pot fi modificate

```
Secventierea activitatilor e necesara -> ne place aceasta cronologie de la
waterfall, dar ceea ce ne deranjeaza e pretul pe care trebuie sa-l platim
,si anume rigiditatea -> o perioada lunga de timp nu mai avem voie sa
schimbam nimic, ex: un proiect pe 2 ani (nu se schimba nimic, decat dupa
finalizarea proiectului, adica dupa acesti 2 ani)
Sparg sistemul in functionalitati si tratez fiecare functionalitate ca pe un
nano-proiect pe care il dezvolt
Buclele de reglaj se intampla mai repede : de mai multe ori intr-un proiect
-> de aceea e mai repede
Procesele iterative : fiecare iteratie construieste o parte din sistemul
real -NU vorbim despre prototipuri cand vine vorba de procesele iterative
!!! -> sistemul are un subsistem de features-uri
Ideea de iterativ si incremental merg impreuna -> la sf fiecarei iteratii,
sistemul meu creste
Ex: am 100 de features , iau primul feature -> ce inseamna primul feature? -
> unul din cele mai importante avantaje e de a prioritize cerintele ->
primul feature inseamna cel mai important feature -> el ajunge sa fie
implementat primul
```



TIME BOXING = Ai un interval de timp fix pentru fiecare iteratie(e rigid, nu ai voie sa il schimbi orice s-ar intampla). Daca NU ai reusit sa implementezi toate feature-urile stabilite, vei scoate din ele, nu vei modifica timpul. E si un bun exercitiu pentru prioritizarea cerintelor

Advantages of Iterative Development

```
1. Clientul primeste cea mai importanta functionalitate la inceput(fiind
primul feature implementat), care va fi, de altfel, si cea mai testata
2. Un risc mai mic de esec
3. Poti prezenta un produs la finalul fiecarui increment
4. Abilitatea de a schimba/ de a te adapta(unul din avantajele principale
ale procesului iterativ)
5. pe durata unei iteratii -> cerintele nu se schimba -> ex:daca facem o
iteratie de 6 luni, e rau si atunci intrebarea e : cat e durata unei
iteratii ? cat de frecvente sunt livrarile unor incrementi? Intrebarile
esentiale in dezvoltarea unui proces
6. livrare accelerata -> sistemul se livreaza repede la client intr-o forma
incipienta -> poate fi modelat si transformat repede
7. Implicarea utilizatorului cu sistemul : E mult mai probabil ca sistemul
sa intalneasca cerintele, iar utilizatorii vor fi mai dedicati sistemului
(un fel de utilizatori beta)

-orice proiect are un risc -> riscul esecului -> cu cat se lungeste bucla de
feedback -> riscul de esec creste
-pe langa avantajele legate de prioritizare, avem si reglarea/optimizarea
procesului in sine
- iteratiile trebuie sa fie cat mai scurte
```

Problems of Iterative Development

```
1. Management problems : progresul poate fi greu de evaluat si problemele
greu de gasit deoarece nu avem o documentatie si o structura stabile care sa
demonstreze ce a fost facut
2. Contractual problems : un contract normal include niste specificatii, dar
fara acestea, trebuie diferite tipuri de contracte (nefiind waterfall cu
ceva fix, nu poti estima de la inceput costul intreg)
    e foarte greu sa gestionezi proiectele de genul pt ca nu stii tot : avem
un proiect cu feature□uri F1, F2, F3, F4, F5 -> poate aparea F7, care nu era
pe lista -> nu stiu de la inceput tot
    imaginea de ansamblu va fi disponibila doar la final
    cum faci un contract? Ce sistem construim? Se mai schimba in timp -> cum
faci contractul? Ce pret pui in contract? Cum stabilim pretul? Nu se mai fac
contracte cu pret fix in general -> nu e in avantaul clientului sa specifice
ce e in fiecare feature pt ca mereu se poate schimba - > daca punem ceva fix
-> se transforma in waterfall -> abandonam pretul fix in contracte si
abordam time and materials , cu accentul pe time-> clientul plateste volumul
```

```
de efort -> nu il putem tepui pe client datorita procesului iterativ
    depinde cat e iteratia : apare forta de a reduce iteratia
    presiunea de a face waterfall vine din conceptul de fix price !!!!
3. Validation problems : fara o specificatie, cum vom stii cum sa testam
sistemul?
4. Maintenance problems : schimbarile continue tind sa corupa structura
software ul si sa l faca mult mai scump pentru a l face sa evolueze incat sa
atinga noile cerinte
- partea de validare se poate rezolva acceptand idea de dynamic, la fel si
la mentenanta
```

The issue of Rework

```
1. Acest tip de dezvoltare poate duce catre schimbarea/ stergerea codului
din iteratiile trecute -> pierderi
- critica de la procesele iterative -> cand integrez F1 cu F2 s-ar putea sa
rescriu anumite bucati din F1 sau sa arunc niste parti etc. -> refacerea
pare ca o pierdere : de ex, motorul nu se potriveste cu caroseria, trebuie
sa refac motorul , sa ajustez ceva, sa remodelez usa etc. -> se pierde
material sau altceva, deci pierdere materiala-> exista si pierdere la
nivelul informatiei -> pierdere de timp -> in sine, nu e o pierdere, ci
exact perfectionarea conceptului
- avantajul e ca ne putem adapta daca pe parcursul dezvoltarii proiectului
daca apar noi cerinte
- acesta e totodata si dezavantaj -> in vederea conceperii unui contract pt
client -> clientul nu stie cum va arata proiectul la final -> ceea ce e un
lucru bun totusi -> trebuie sa lasi flexibila evolutia proiectului -> nu
stii care va fi exact forma lui finala

2. E mai bine sa refaci decat sa peticesti cod prost scris
```

Supported by techniques :
1. Refactoring
2. Regression Test
3. Continous Integration

Predictive Planning

1. Faci un plan (greu de anticipat -> pret si scope fix)
2. Urmaresti planul (usor de anticipat)

Problema : de foarte multe ori se schimba planurile

2 moduri sa gestionezi cerintele in schimbare :
1. O mai buna analiza a cerintelor
2. Imbratiseaza schimbarea :)

Adaptive Planning

Merge pe ideea ca este imposibil sa prezici ce se va intampla

SW este controlabil dar nu predictibil

Nu exista "conform planului" in aceasta metoda, exista un plan, dar e doar de suprafata

Are pret fix, dar scope variabil

Diferente : in predictiva (fac un plan si-l urmez, dar e foarte greu sa prezici -> fix price, fix scope), in adaptive(nu mergem rigid dupa un plan, faci mai mult planning decat in predictiva -> fix price , variable scope -> ceea ce platesti e time and materials -> sa fim cat mai rapid adaptabili -> livrare de SW cat mai repede-> mai multe iteratii si mai scurte -> cu cat sunt mai scurte, cat atat bucla de reglaj se manifesta mai rapid)

Agile methods

Neintelegerile cu sefii legate de metodele de design au dus la crearea acestor metode agile

```
1. Sunt focusate pe cod mai mult decat pe un preset design
2. au o abordare iterativa
3. livreaza SW rapid si il dezvolta rapid pt a intalni cerintele
```

Agile Manifesto

We are uncovering better ways of developing software by doing it and helping others do it.

Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Principles of agile methods

- **Customer satisfaction** by rapid, continuous delivery of useful software
- Software is **delivered frequently** (weeks rather than months)
- **Working software** is the principal measure of progress
- Even **late changes** in requirements are welcomed
- **Close (daily) cooperation** between business people and developers
- **Colocation**: Face-to-face conversation is the best form of communication
- **Trust**: build around motivated individuals, who should be trusted
- Continuous attention to **technical excellence** and good design
- **Simplicity**
- **Self-organizing teams**

```
1. Preferam sa punem accentul pe indivizi, pe capacitatea lor si pe
interactiunile dintre ei pt a dezvolta un sistem SW
2. Apreciem un SW functional si livrarea constanta de functionalitate fata
de documentatii stufoase
3. Contractele -> cum se negociaza? E mai importanta colaborarea vie cu
clientul -> opus cu waterfall (clientul vine, cere si apoi vine la final sa
ia proiectul)
4. Nu e important ca design-ul sa fie perfect, ci e important sa avem
abilitatea de a ne schimba, de a fi agili
5. Nu putem executa teste pt ca nu avem cod , dar putem scrie teste fara sa
avem cod -> in cazul asta, testele devin un mod formal prin care definim
cerintele
6. Clientul trebuie sa se implice in partea de dezvoltarea proiectului -> se
exprima bine in SCRUM
7. Dezavantaje metode agile : ➔ sunt foarte mult centrate pe individ si pe
echipe coezive -> e foarte greu sa scalezi
8. Alt dezavantaj : nu exista o imagine de ansamblu, sudarea echipei
```

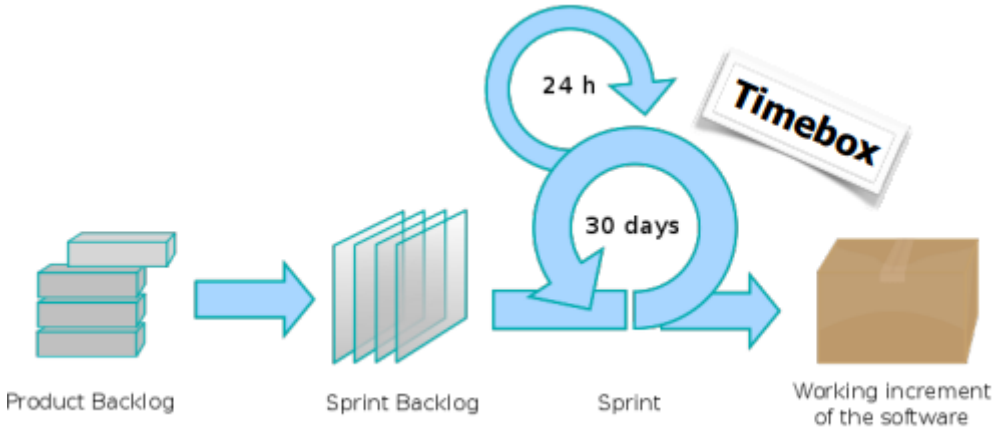
Scrum

- E o metoda Agile (e un proces iterativ cu iteratii cat mai scurte numite **sprint-uri**)
- Ce e un sistem agil? E un sistem care favorizeaza schimbarea, se refera la planificarea a ce se va construi in urmatoarele saptamani, apoi se adapteaza de la acel punct
- Cascada : planificam totul in avans apoi urmam planul pe o durata fixa stabilita la inceput (ex:1 an, 2 ani etc.)
- Un feature este scris din perspectiva utilizatorului final : **As a (role), I want (feature) so that (benefit)**
- Feature-urile sunt cunoscute ca **user-stories**

Scrum Roles

1. Scrum Master (the Project Manager)
   - Este cumva intre primele 2 roluri
   - Este un membru din echipa (un fel de aparator al procesului) ce se asigura ca sunt respectati toti pasii si ca proiectul progreseaza corespunzator
   - Are grija ca procesul sa fie urmat intocmai, urmareste evolutia task-urilor si a efortului depus de echipa pe un burndown chart
   - Are grija ca fiecare membru sa aiba toate instrumentele necesare
   - Programeaza intalnirile
2. Product Owner (represents the stakeholders -> basically clientii)
   - Persoana careia ii pasa cel mai mult de produs, e mereu atent ca lucrurile care se construiesc si feature-urile care se adauga sa fie mereu cele mai valoroase -> in asa fel incat sa fie la orice moment maximizam valorile pe care sistemul SW le are
   - La fiecare moment, scopul product owner-ului e sa se asigure ca lucrurile care se construiesc maximizeaza valoarea produsului -> adica : ca se aleg mereu feature-urile cele mai importante, ca se revizuiesc prioritatile astfel incat la fiecare noua iteratie sa se obtina cea mai mare valoare posibila
   - Se ocupa ca prioritatile produsului sa fie respectate de echipa care le va implementa
   - Are imaginea de ansamblu asupra produsului, ea da directia produsului
3. Team (echipa de dezvoltare)
   - self organizing group : asta e idea centrala din procesele agile : echipele au autonomie , adica ele decid cum sa livreze acele bucati de functionalitate, respectand procesul, respectand prioritatile clientului (prioritatile product owner-ului) -> mai apoi echipa se organizeaza ea intern cum sa faca livrarea asta cel mai bine
   - echipa actioneaza autonom, tinand cont doar de prioritatile date de product owner
4. Other Roles :
   1. Tester -> se asigura ca merge bine proiectul
   2. Customer -> cel care foloseste si plateste

Scrum process

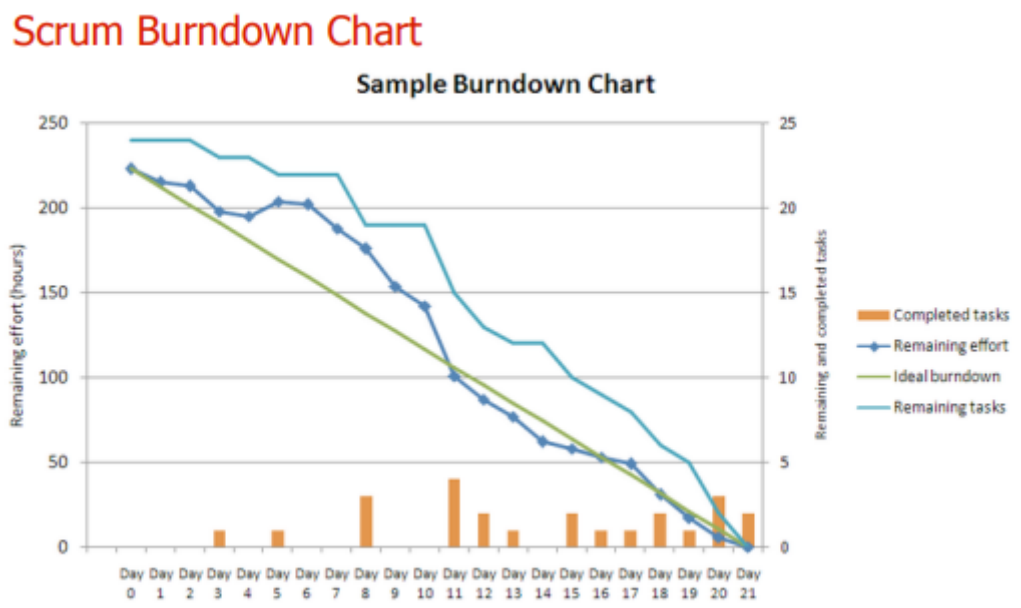


Product Backlog = format din toate user-stories (este ca un wish list)
Sprint Backlog = ce se va implementa intr-un sprint

PASI:

```
1. Pleaca cumva de la PRODUCT OWNER -> el primeste cerintele si directia de
la client si pe baza acestuia se face o lista de prioritati(Product Backog)
2. Cu alte cuvinte, product owner-ul trebuie sa spuna care sunt feature-
urile care trebuiesc implementate la fiecare moment (la sfarsitul fiecarei
iteratii , adica foarte des) si apoi sa le prioritizeze -> sa actualizeze de
fiecare data lista cu prioritati
3. Apoi echipa (care e autonoma) sa-si calculeze o viteza/capacitate de
lucru si isi preia, in functie de asta (de viteza de lucru), niste item-uri
din lista prioritizata (se mai numeste si product backlog) -> product
backlog = stiva in care sunt puse lucrurile care trebuie facute, care sunt
de facut -> aceasta e mereu revizuita > echipa isi alege cele mai prioritare
item-uri, task-uri din acest product backlog si formeaza un sprint backlog
4. Cum face asta? Printr-un meeting de planificare la inceputul unui sprint-
> in prima zi a unui sprint se face planning-ul
5. Planning-ul inseamna : ne uitam la ce avem in product backlog impreuna cu
product owner-ul si stabilim un subset de feature-uri de aici in ordinea
prioritatilor avand in vedere capacitatea echipei la momentul ala sa faca ->
se construieste astfel un sprint backlog
6. Sprint-ul e numele pe care SCRUM l-a dat unei iteratii -> SPRINT = O
ITERATIE > de ce ii spun sprint? Ca sa iti sugereze idea de rapid (sprintul
dureaza intre 1-4 saptamani)
7. Despart feature-urile in niste task-uri -> se impart apoi task-urile
intre membrii echipei
8. Fin dorinta de a reduce riscurile -> avem o iteratie in iteratie -> adica
echipa face o revizitare asupra lucrurilor pe care le-au facut cu o zi in
urma (prin daily stand-up meeting) -> ca sa evitam blocajele -> avem o bucla
de reglaj in interiorul unei bucle de reglaj
9. Dupa ce se termina cele 1-4 saptamani si daily-uri -> se termina un
increment(sprint)
10. Cand se termina aceasta bucata de functionalitate se fac 2 lucruri -> se
face un demo al incrementului/sprintului respectiv (product owner-ul
impreuna cu echipa fac sprint review) -> se da feedback -> se mai face si o
retrospectiva a sprintului (ce a mers bine/gresit , unde s-a incetinit
procesul etc. -> pot fi destul de tensionate)
11. De ex: daca am zis ca putem sa facem n feature-uri in x timp, de ce am
facut doar 80% din ele? Unde ne-am inselat? Pt ca ori s-a intamplat ceva
neprevazut, ori am asteptat dupa altii, ori a fost un rist tehnologic pe
care nu l-am anticipat -> se fac aceste retrospective ca sa imbunatatim
procesul de dezvoltare al feature-ului urmator
```

Scrum Burndown Chart



Ne spune cand ar fi gata un sprint si detalii despre el

```
➢ E un graphic in care noi am spus ca facem x task-uri -> e BURNDOWN pt ca
vrem sa ardem efortul ala, sa consumam task-urile pe care trebuie sa le
facem din sprint backlog
 ➢ initial avem de ex 23 de task-uri de facut in 21 de zile-> dupa o zi
cate avem?
  ➢ se poate si sa urcam dupa ce am coborat dupa un task -> asta vine dupa
niste teste -> daca echipa de testare a spus ca task-ul nu e inca finalizat,
crestem inapoi in graphic de unde am coborat initial
   ➢ tot ceea ce conteaza este sa avem un feedback zilnic -> trebuie sa
vezi constant ce se intampla, nu doar la sf unei iteratii
```

Concluzii

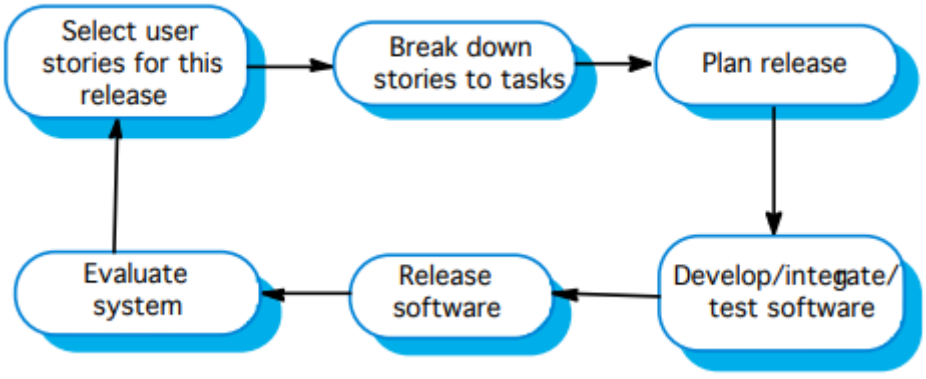
```
➢ sprint backlog-ul e ca un inventar al task-urilor care urmeaza sa fie
implementate in respectivul sprint, iar task-urile se iau din product
backlog, care e o stiva
➢ daily standup -> sunt folosite pt a avea o bucla de reglaj in interiorul
buclei de reglaj care e deja sprintul, pt a vedea daca sunt blocaje ca sa
poata fi rezolvate etc.
➢ un sistem SW nu e predictibil, dar poate fi planificabil
➢ clientul poate modifica oricand product backlog-ul, poate sa
reprioritizeze, poate sa stearga sau sa adauge task-uri/feature-uri -> acest
product backlog-ul nu e static -> e dynamic
➢ cea mai problematica activitate -> estimarea efortului
➢ cat incape intr-un sprint? Trebuie sa avem o unitate de masura a
efortului -> pt ca un task poate fi mai mare sau mai mic, un feature poate
fi mai complex sau nu samd. -> nu se poate spune un nr de task-uri -> cand
stim unde sa ne oprim? Cat putem pune? -> unitatea de masura va fi ora =>
```

```
cate ore va petrece un membru al echipei pt task-ul respective -> trebuie sa
ne gandim la mult mai multe aspecte ➢ avem 8 oameni in 10 zile -> 80 de
parti in base * 8 ore -> 640 ore capacitate de lucru pt echipa
➢ daca nu suntem toti full time? -> trebuie sa fim mai realisti, poate avem
500 de ore capacitatea de lucru
➢ din cele 10 zile -> de ex 1 zi e pt planning meeting -> trebuie sa tinem
cont de mai multe aspecte
➢ daca vrem sa luam un task din product backlog, cam cate person hours ar
costa? Nu avem de unde sa stim -> avantajul e ca feature-urile pe care le
punem in product backlog sunt definite cu granularitate a. i. sa se poata
estima timpul de lucru -> exista riscul sa gresim volumul de effort necesar
pt a implementa un task
➢ daca nu se poate estima -> task-ul e prea mare si trebuie spart
➢ cum noi facem estimarile pe un sprint (iteratie de 1-4 sapt maxim) ->
maxim ce putem gresi e un sprint
➢ story points = descrierea unui anumit feature -> se ia un feature/story,
il spargem in task-uri (sunt mai usor de implementat, de ex intr-o zi) ->
story-urile se spartg in task-uri -> planificam task-urile si le estimam, le
implementam, le evaluam si luam urmatorul story samd.
➢ Unde se planifica task-urile? Intr-un task management system, de ex JIRA
```

Extreme programming(XP)

Cea mai buna metoda Agile cu o abordare extrema a metodei iterative
1. Cate o noua versiune in fiecare zi/ noapte
2. Incrementii sunt livrati clientilor la fiecare 2-3 saptamani
3. Toate testele vor fi rulate pentru fiecare build -> un build este acceptat doar daca trece toate testele

The XP release cycle



The story within XP Story Cards

Date: _____ Story Nr: _____	Type of Activity: New ☐ Fix ☐ Enhance ☐ Funct.Testing ☐ Priority: User _____ Technical _____ Risk: _____ Technical Estimate: _____
Story Name:	
Story Description:	
Notes:	

Task Tracing

Date	Status	ToDo	Comments

Extreme programming practices

Incremental planning	Requirements are recorded on Story Cards and the Stories to be included in a release are determined by the time available and their relative priority. The developers break these Stories into development 'Tasks'.
Small Releases	The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.
Simple Design	Enough design is carried out to meet the current requirements and no more.
Test first development	An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.

Black-box testing

Pair Programming	Developers work in pairs, checking each other's work and providing the support to always do a good job.
Collective Ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers own all the code. Anyone can change anything.
Continuous Integration	As soon as work on a task is complete it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.
Sustainable pace	Large amounts of over-time are not considered acceptable as the net effect is often to reduce code quality and medium term productivity
On-site Customer	A representative of the end-user of the system (the Customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

When doesn't XP work?

- Business Culture
  - Big forehead specifications
  - Obsession with writing useless documents
- Extra-work to prove commitment to the company
  - instead of just going for a 40h work-week... no extra-time!
- Size-matters
  - doesn't work with 100 programmers
  - only <= 10 programmers
- Physical separation
  - even separate floor kills XP
- Environments with high viscosity
  - large build times, impossible to run all tests etc.

Concluzii

```
➢ Pt fiecare task avem prioritatea din perspectiva user-ului (clientului)
si din perspectiva tehnica(echipa de dezvoltare)
➢ De ce trebuie sa avem si prioritate din perspectiva tehnica? Pt ca si ea
trebuie sa aiba o prioritizare a task-urilor ca sa construiasca sistemul cat
mai efficient si logic dpdv tehnic
➢ Pe baza prioritatilor se mai estimeaza un risc -> ca ei sa esueze -> se
ia ca unitate de masura virtuala points-urile -> T-shirt sizes : XS, S, M,
L, XL (se folosesc pt estimare)
➢ De ex, nu dau estimarea in ore, zic ca e un T-shirt size L , ceea ce
inseamna 7 story points
➢ La sf unui sprint, nu zicem chiar nr de task-uri facute, spunem cate
story-points a facut echipa, de ex 21 de story points -> velocity (viteza
echipei -> volumul de effort pe care echipa il poate face intr-un sprint)
➢ De ex, cand se negociaza un contract, spunem ca echipa poate face 20 de
story points -> estimez pt primele 6 sprint-uri/iteratii ca voi face 20 de
story points cand negociez -> apoi reglam/stabilizam (poate la o echipa
story point-ul e mai mare, la alta mai mic etc.) -> stabilizam velocity-ul
la 20 story points
➢ Deci, pe un burndowm chart stabilim atat nr de task-uri cat si efortul si
il urmarim zilnic-> de ex, am terminat un task de 5 story points , mai avem
17 si tot asa -> idea e sa te indrepti spre tinta
➢ Daca nu ajungi -> afli repede ca nu ajungi si poti sa revizuiesti
velocity-ul, poti sa redefinesti L, S, XS etc. -> ca sa avem acuratete (sa
fie cat mai apropiat de realitate)
➢ Procese iterative -> TIME BOXING -> daca ceva e fix, atunci asta e durata
iteratiei!! -> nu inseamna ca esuam, ci ca intelegem cat putem face -> la sf
iteratiei constatam ca nu am facut niste story point-uri -> nimeni nu spune
```

```
ca la sf unui sprint D si E vor fi ceea ce intra in sprint-ul urmator : de
ex reusim sa facem A,B si C si era planificat sa facem si D si E -> probabil
se vor face in sprintul urmator, dar probabil nu e si CERT!!!
➢ Clientul are posibilitate sa reevalueze prioritatile task-urilor la sf
fiecarui sprint -> atunci in meetingul de planning pt urmatorul sprint se ia
de la 0 tot, adica D si E nu vor fi automat incluse in noul sprint si nici
in product backlog, ele se vor intoarce in product backlog si bineinteles,
va fi revizuita prioritatea lor de catre client
➢ Atentie, nu se prelungeste niciodata durata de viata a unei
iteratii/sprint pt ca durata fixa si scurta a acestuia creeaza mai multe
oportunitati de reglaj, de reevaluare, de reestimare si de reprioritizare
```

Risk aware processes

The Rational Unified Process

- A modern process model derived from the work on the UML and associated process.
- Normally described from 3 perspectives:
  - **Dynamic** perspective: **phases** over time;
  - **Static** perspective: **process activities**;
  - **Practice** perspective: suggests good practice.

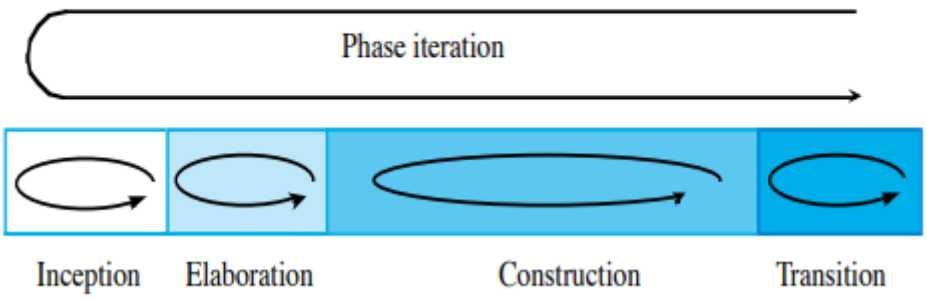
Incremental + Iterative + Risk Aware

Procese menite sa estompeze riscurile

```
➢ Si procesele agile sunt risk aware -> ele estompeaza riscurile prin
faptul ca se poate intampla orice intr-un sprint, dar apoi oricat de
catastrofal ar fi fost sprintul-> e doar un sprint
➢ Unul din procesele risk aware e RUP -> process de dezvoltare mai
birocratic -> are avanajul ca e mai usor de contractat, de planificat
aparent
➢ Cel mai mare risc pe care si-l asuma waterfall-ul e ca nu am ajuns unde
trebuie, chiar daca procesul a fost minunat, rezultatul e groaznic,
dezamagitor pt client
➢ RUP nu e waterfall -> incearca sa aduca aici sentimentul de progres de la
waterfall
➢ De ce spun sentiment de progres? Pt ca de ex, la procesele iterative nu
putem pune progress bar : de ex, uite asta e sistemul si am ajuns la 20%,
dar nu putem face un progress bar pt ca nu stim cat e tot proiectul , pe
```

```
cand la waterfall stim cat e proiectul
➢ Sigur, ca progress la procesele iterative pot spune ca avem progress, dar
nu il pot manifesta numeric, sub forma de procente pt ca nu stim cat mai
avem pana la final pt ca backlog-ul se modifica constant dupa fiecare
iteratie
➢ Un domeniu in care se poate folosi progress bar-ul si in care avem
waterfall sunt de ex in constructii -> de ex, construim o casa -> tot ce
foloseste materie si construieste produse poate avea progress bar ->
fundatie e 5% etc.
```

RUP phase model



RUP Dynamic Perspective : Phases

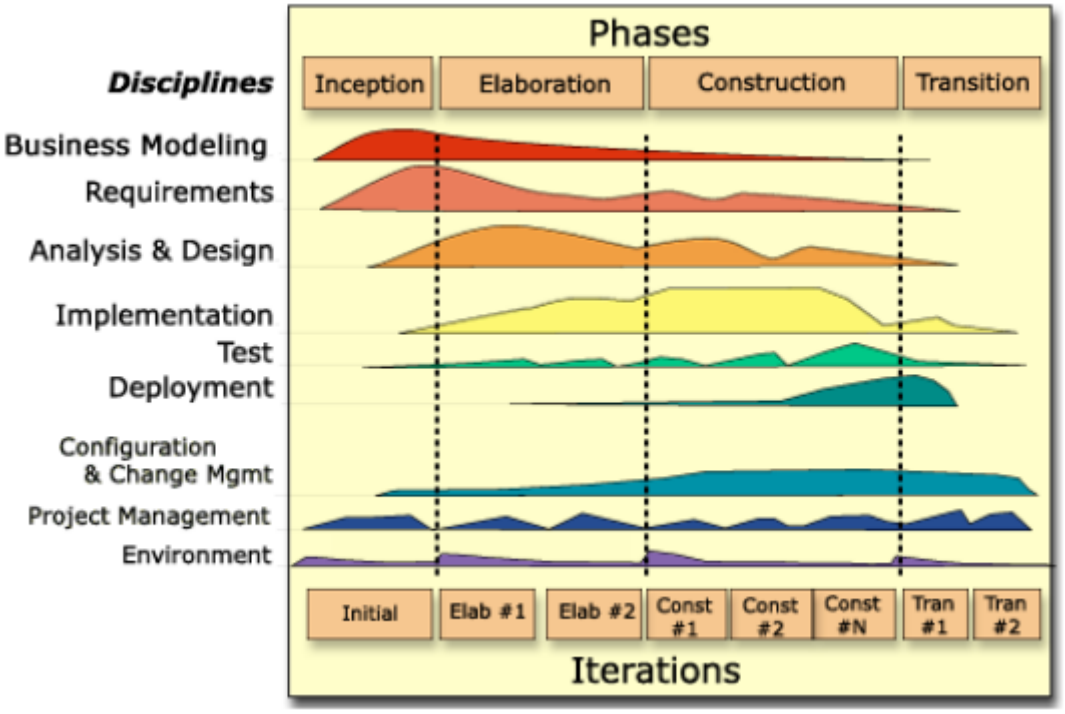
- Inception
  - Establish the business case for the system.
- Elaboration
  - Develop an understanding of the problem domain and the system architecture.
- Construction
  - System design, programming and testing.
- Transition
  - Deploy the system in its operating environment.

```
La RUP se incearca un model hybrid, iar pt asta avem 4 faze, nu se intretesc
:
    o Inception : se defineste la ce urmeaza la fie folosit sistemul
    o Elaborare : se specifica riguros cerintele, se vine cu o arhitectura,
se face un concept general
    o Constructie : se face implementarea, testarea etc.
    o Tranzition : produsul trece in zona operationala si incepe sa
functioneze in productie -> produsul devine operational

➢ Pare ca e waterfall 100% -> dar in interiorul fiecarei faze se fac multe
```

```
iteratii , se lucreaza iterativ
➢ Faza nu e totuna cu activitatea !!!! -> la waterfall prima activitate
facuta e captarea cerintelor -> ce activitate facem? Captarea cerintelor ->
diferenta e ca intr-o faza se executa mai multe activitati -> in fiecare
faza se executa mai mult sau mai putin un subset din activitatile tipice de
dezvoltare SW
➢ Fazele nu implica un waterfall, ci doar o secventa de milestone-uri, dar
inteteserea activitatilor se intampla ca la procesele iterative
➢ Ideea de matrice de activitati/faza -> intr-o faza pot sa am, cu ponderi
diferite , mai multe activitati si o faza nu e ceva ce se executa unitary in
bloc, ci se executa iterativ, iar in fiecare iteratie se reevalueaza etc.,
ca in procesee iterative
```

RUP Phases-Activities Matrix



RUP Dynamic Perspective: Activities/Workflows/Disciplines

Workflow	Description
Business modelling	The business processes are modelled using business use cases.
Requirements	Actors who interact with the system are identified and use cases are developed to model the system requirements.
Analysis and design	A design model is created and documented using architectural models, component models, object models and sequence models.
Implementation	The components in the system are implemented and structured into implementation sub-systems. Automatic code generation from design models helps accelerate this process.
Test	Testing is an iterative process that is carried out in conjunction with implementation. System testing follows the completion of the implementation.
Deployment	A product release is created, distributed to users and installed in their workplace.
Configuration and change management	This supporting workflow managed changes to the system
Project management	This supporting workflow manages the system development
Environment	This workflow is concerned with making appropriate software tools available to the software development team.

RUP good practice

1. Develop software iteratively
2. Manage requirements
   - explicit management: use-cases
3. Visually model software
   - UML diagrams
4. Verify quality of software
5. Control changes to software
   - make use of CMS (Control Management Systems) for versioning, build and integration

```
➢ De la RUP (rational unify process -> mai invechit, dar inca folosit) am
invatat ca el a impus dezvoltarea iterativa, am invatat sa captam mai bine
cerintele (usecase-uri), am invatat sa modelam SW folosind diagrame UML
(unify process -> au facut un model unficat, rational -> asa se numea firma)
➢ RUP e mai lent decat SCRUM -> nu e waterfall nici el, in ciuda cascadarii
de faze care aduce a waterfall, faza nu se suprapune cu activitatea -> in
interiorul unei faze pot sa ai mai multe iteratii si mai multe activitati,
cu ponderi diferite -> RUP e un process iterativ si incremental, pt ca
sistemul se construieste incremental -> se spune ca e risc meet aware
```

```
process din felul in care se fac iteratiile sau verificare la sf fiecarei
iteratii se estompeaza riscurile
```